

Our code thinks that all these locations refer to invalid words. However, if you look at some of the words referred to here, they look pretty valid. For example, `text[103:106]` outputs the valid word `and`. The reason our code thinks that `and` is an invalid word is that it isn't in our word list. Of course, we could add it to our word list manually and continue using that approach as we need our code to recognize words. But remember that we want this space insertion algorithm to work for millions of pages of scanned text, and they may contain many thousands of unique words. It would be helpful if we could import a word list that already contained a substantial body of valid English words. Such a collection of words is referred to as a *corpus*.

Using an Imported Corpus to Check for Valid Words

Luckily, there are existing Python modules that allow us to import a full corpus with just a few lines. First, we need to download the corpus:

```
import nltk
nltk.download('brown')
```

We've downloaded a corpus called `brown` from the module called `nltk`. Next, we'll import the corpus:

```
from nltk.corpus import brown
wordlist = set(brown.words())
word_list = list(wordlist)
```

We have imported the corpus and converted its collection of words into a Python list. Before we use this new `word_list`, however, we should do some cleanup to remove what it thinks are words but are actually punctuation marks:

```
word_list = [word.replace('*', '') for word in word_list]
word_list = [word.replace('[', '') for word in word_list]
word_list = [word.replace(']', '') for word in word_list]
word_list = [word.replace('?', '') for word in word_list]
word_list = [word.replace('.', '') for word in word_list]
word_list = [word.replace('+', '') for word in word_list]
word_list = [word.replace('/', '') for word in word_list]
word_list = [word.replace('; ', '') for word in word_list]
word_list = [word.replace(':', '') for word in word_list]
word_list = [word.replace(',', '') for word in word_list]
```
